

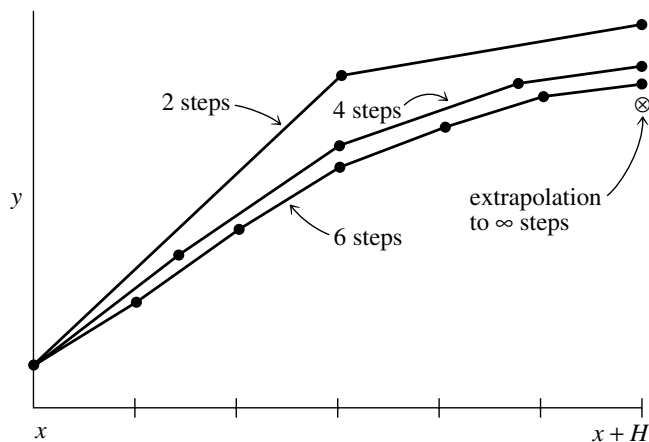
- Forsythe, G.E., Malcolm, M.A., and Moler, C.B. 1977, *Computer Methods for Mathematical Computations* (Englewood Cliffs, NJ: Prentice-Hall).
- Dormand, J.R., and Prince, P.J. 1986, "Runge-Kutta Triples," *Computers and Mathematics with Applications*, vol. 12A, pp. 1007–1017.[3]
- Shampine, L.F. 1986, "Some Practical Runge-Kutta Formulas," *Mathematics of Computation*, vol. 46, pp. 135–150.[4]
- Hairer, E., Nørsett, S.P., and Wanner, G. 1993, *Solving Ordinary Differential Equations I. Nonstiff Problems*, 2nd ed. (New York: Springer). Fortran codes at <http://www.unige.ch/~hairer/software.html>. [5]
- Gustafsson, K. 1991, "Control Theoretic Techniques for Step Size Selection in Explicit Runge-Kutta Methods," *ACM Transactions on Mathematical Software*, vol. 17, pp. 533–554.[6]
- Hairer, E., Nørsett, S.P., and Wanner, G. 1996, *Solving Ordinary Differential Equations II. Stiff and Differential-Algebraic Problems*, 2nd ed. (New York: Springer), p. 28.[7]
- Söderlind, G. 2003, "Digital Filters in Adaptive Time-stepping," *ACM Transactions on Mathematical Software*, vol. 29, pp. 1–26.[8]
- Numerical Recipes Software 2007, "Routine Implementing an Eighth-order Runge-Kutta Method," *Numerical Recipes Webnote No. 20*, at <http://www.nr.com/webnotes?20> [9]

## 17.3 Richardson Extrapolation and the Bulirsch-Stoer Method

The techniques in this section are for differential equations containing smooth functions. With just three caveats, we believe that the Bulirsch-Stoer method, discussed here, is the best-known way to obtain high accuracy solutions to ordinary differential equations with minimal computational effort. The caveats are these:

- If you have a nonsmooth problem, for example, a differential equation whose right-hand side involves a function that is evaluated by table look-up and interpolation, go back to Runge-Kutta with an adaptive stepsize choice. That method does an excellent job of feeling its way through rocky or discontinuous terrain. It is also an excellent choice for a quick-and-dirty, low accuracy solution of a set of equations.
- The techniques in this section are not particularly good for differential equations that have singular points *inside* the interval of integration. A regular solution must tiptoe very carefully across such points. Runge-Kutta with adaptive stepsize can sometimes effect this; more generally, there are special techniques available for such problems, beyond our scope here but touched on in §18.6.
- There *may* be a few problems that are both very smooth and have right-hand sides that are very expensive to evaluate, for which predictor-corrector methods, discussed in §17.6, are the methods of choice.

The methods in this section involve three key ideas. The first is *Richardson's deferred approach to the limit*, which we already met in §4.3 on Romberg integration. The idea is to consider the final answer of a numerical calculation as itself being an analytic function (if a complicated one) of an adjustable parameter like the stepsize  $h$ . That analytic function can be probed by performing the calculation with various values of  $h$ , *none* of them being necessarily small enough to yield the accuracy that we desire. When we know enough about the function, we *fit* it to some analytic form



**Figure 17.3.1.** Richardson extrapolation as used in the Bulirsch-Stoer method. A large interval  $H$  is spanned by different sequences of finer and finer substeps. Their results are extrapolated to an answer that is supposed to correspond to infinitely fine substeps. In the Bulirsch-Stoer method, the integrations are done by the modified midpoint method, and the extrapolation technique is polynomial extrapolation.

and then *evaluate* it at that mythical and golden point  $h = 0$  (see Figure 17.3.1). Richardson extrapolation is a method for turning straw into gold! (Lead into gold for alchemist readers.)

The second idea has to do with what kind of fitting function is used. Bulirsch and Stoer first recognized the strength of *rational function extrapolation* in Richardson-type applications. That strength is to break the shackles of the power series and its limited radius of convergence, out only to the distance of the first pole in the complex plane. Rational function fits can remain good approximations to analytic functions even after the various terms in powers of  $h$  all have comparable magnitudes. In other words,  $h$  can be so large as to make the whole notion of the “order” of the method meaningless — and the method can still work superbly. Nevertheless, more recent experience suggests that for smooth problems straightforward polynomial extrapolation is slightly more efficient than rational function extrapolation. (This may tell us more about the kinds of problems used for tests than about the methods themselves.) In any event, we will adopt polynomial extrapolation as our default. You might wish at this point to review §3.2, where polynomial function extrapolation was already discussed.

The third idea is to use an integration method whose error function is strictly *even*, allowing the rational function or polynomial approximation to be in terms of the variable  $h^2$  instead of just  $h$ . We will expand on this idea in the next subsection, on the modified midpoint method.

Put these ideas together and you have the *Bulirsch-Stoer method* [1]. A single Bulirsch-Stoer step takes us from  $x$  to  $x + H$ , where  $H$  is supposed to be quite a large — not at all infinitesimal — distance. That single step is a grand leap consisting of many (e.g., dozens to hundreds) substeps of the modified midpoint method, which are then extrapolated to zero stepsize.

### 17.3.1 Modified Midpoint Method

The *modified midpoint method* advances a vector of dependent variables  $y(x)$  from a point  $x$  to a point  $x + H$  by a sequence of  $n$  substeps each of size  $h$ ,

$$h = H/n \quad (17.3.1)$$

In principle, one could use the modified midpoint method in its own right as an ODE integrator. In practice, the method finds its most important application as a part of the more powerful Bulirsch-Stoer technique.

The number of right-hand side evaluations required by the modified midpoint method is  $n + 1$ . The formulas for the method are

$$\begin{aligned} z_0 &\equiv y(x) \\ z_1 &= z_0 + hf(x, z_0) \\ z_{m+1} &= z_{m-1} + 2hf(x + mh, z_m) \quad \text{for } m = 1, 2, \dots, n-1 \\ y(x + H) &\approx y_n \equiv \frac{1}{2}[z_n + z_{n-1} + hf(x + H, z_n)] \end{aligned} \quad (17.3.2)$$

Here the  $z$ 's are intermediate approximations that march along in steps of  $h$ , while  $y_n$  is the final approximation to  $y(x + H)$ . The method is basically a “centered difference” or “midpoint” method (compare equation 17.1.2), except at the first and last points. Those give the qualifier “modified.”

The modified midpoint method is a second-order method, like (17.1.2), but with the advantage of requiring (asymptotically for large  $n$ ) only one derivative evaluation per step  $h$  instead of the two required by second-order Runge-Kutta.

The usefulness of the modified midpoint method to the Bulirsch-Stoer technique (§17.3) derives from a “deep” result about equations (17.3.2), due to Gragg. It turns out that the error of (17.3.2), expressed as a power series in  $h$ , the stepsize, contains only *even* powers of  $h$ ,

$$y_n - y(x + H) = \sum_{i=1}^{\infty} \alpha_i h^{2i} \quad (17.3.3)$$

where  $H$  is held constant but  $h$  changes by varying  $n$  in (17.3.1). The importance of this even power series is that, if we play our usual tricks of combining steps to knock out higher-order error terms, we can gain *two* orders at a time!

For example, suppose  $n$  is even, and let  $y_{n/2}$  denote the result of applying (17.3.1) and (17.3.2) with half as many steps,  $n \rightarrow n/2$ . Then the estimate

$$y(x + H) \approx \frac{4y_n - y_{n/2}}{3} \quad (17.3.4)$$

is *fourth-order* accurate, the same as fourth-order Runge-Kutta, but requires only about 1.5 derivative evaluations per step  $h$  instead of Runge-Kutta's four evaluations. Don't be too anxious to implement (17.3.4), since we will soon do even better.

Now would be a good time to look back at the routine `qsimp` in §4.2, and especially to compare equation (4.2.4) with equation (17.3.4) above. You will see that the transition in Chapter 4 to the idea of Richardson extrapolation, as embodied in Romberg integration of §4.3, is exactly analogous to the transition in going from this section to the next one.

A routine that implements the modified midpoint method will be given as part of the implementation of `StepperBS`, in the `dy` member function.

### 17.3.2 The Bulirsch-Stoer Method

Consider attempting to cross the interval  $H$  using the modified midpoint method with increasing values of  $n$ , the number of substeps. Bulirsch and Stoer originally proposed the sequence

$$n = 2, 4, 6, 8, 12, 16, 24, 32, 48, 64, 96, \dots, [n_j = 2n_{j-2}], \dots \quad (17.3.5)$$

More recent work by Deuffhard [2,3] suggests that the sequence

$$n = 2, 4, 6, 8, 10, 12, 14, \dots, [n_j = 2(j + 1)], \dots \quad (17.3.6)$$

is usually more efficient. For each step, we do not know in advance how far up this sequence we will go. After each successive  $n$  is tried, a polynomial extrapolation is attempted. That extrapolation gives both extrapolated values and error estimates. If the errors are not satisfactory, we go higher in  $n$ . If they are satisfactory, we go on to the next step and begin anew with  $n = 2$ .

Of course there must be some upper limit, beyond which we conclude that there is some obstacle in our path in the interval  $H$ , so that we must reduce  $H$  rather than just subdivide it more finely. Moreover, precision loss sets in if we choose too fine a subdivision. In the implementations below, the maximum number of  $n$ 's to be tried is called KMAXX. We usually take this equal to 8; the eighth value of the sequence (17.3.6) is 16, so this is the maximum number of subdivisions of  $H$  that we allow.

We enforce error control, as in the Runge-Kutta method, by monitoring internal consistency and adapting the stepsize to match a prescribed bound on the local truncation error. Each new result from the sequence of modified midpoint integrations allows a tableau like that in equation (3.2.2) to be extended by one additional set of diagonals. Write the tableau as a lower triangular matrix:

$$\begin{array}{cccc} T_{00} & & & \\ T_{10} & T_{11} & & \\ T_{20} & T_{21} & T_{22} & \\ \dots & \dots & \dots & \dots \end{array} \quad (17.3.7)$$

Here  $T_{k0} = y_k$ , where  $y_k$  is  $y(x_n + H)$  computed with the stepsize  $h_k = H/n_k$ . Neville's algorithm, equation (3.2.3), with  $P$  replaced by  $T$ ,  $x_i = h_i^2$ , and  $x = 0$ , can be written

$$T_{k,j+1} = T_{kj} + \frac{T_{kj} - T_{k-1,j}}{(n_k/n_{k-j})^2 - 1} \quad j = 0, 1, \dots, k-1 \quad (17.3.8)$$

Each new stepsize  $h_i$  starts a new row in the tableau, and then the polynomial extrapolation fills the rest of the row. Each new element in the tableau comes from the two closest elements in the previous column. Elements in the same column have the same order, and  $T_{kk}$ , the last element in each row, is the highest-order approximation with that stepsize. The difference between the last two elements in a row is taken as the (conservative) error estimate. How should we use this error estimate to adjust the stepsize? A good strategy was originally proposed by Deuffhard [2,3]. We will use a modified version [4], next described.

### 17.3.3 Stepsize Control Algorithm for Bulirsch-Stoer

The elements in the tableau are actually vectors corresponding to the vector  $y$  of dependent variables. Accordingly, define

$$\text{err}_k = \|T_{kk} - T_{k,k-1}\| \quad (17.3.9)$$

where the norm is the same scaled norm used in equation (17.2.9). Error control is enforced by requiring  $\text{err}_k \leq 1$ .

Now  $T_{kk}$  is of order  $2k + 2$  and  $T_{k,k-1}$  is of order  $2k$ , which is therefore the order of  $\text{err}_k$ . In other words,

$$\text{err}_k \sim H^{2k+1} \quad (17.3.10)$$

Thus a simple estimate of a new stepsize  $H_k$  to obtain convergence in a fixed column  $k$  would be (cf. equation 17.2.12)

$$H_k = HS_1 \left( \frac{S_2}{\text{err}_k} \right)^{1/(2k+1)} \quad (17.3.11)$$

where  $S_1$  and  $S_2$  are safety factors smaller than one.

Which column  $k$  should we aim to achieve convergence in? Let's compare the work required for different  $k$ . Suppose  $A_k$  is the work to obtain row  $k$  of the extrapolation tableau. Assume the work is dominated by the cost of evaluating the functions defining the right-hand sides of the differential equations. For  $n_k$  subdivisions in  $H$ , the number of function evaluations can be found from the recurrence

$$\begin{aligned} A_0 &= n_0 + 1 \\ A_{k+1} &= A_k + n_{k+1} \end{aligned} \quad (17.3.12)$$

The work per unit step to get column  $k$  is therefore

$$W_k = \frac{A_k}{H_k} \quad (17.3.13)$$

The optimal column index is the one that minimizes  $W_k$ . The strategy is to set a target  $k$  for the next step, and then choose the stepsize from (17.3.11) to try to get convergence (i.e.,  $\text{err}_k \leq 1$ ) for that value of  $k$  on the next step.

In practice, you compute the extrapolation tableau (17.3.7) row by row, but only test for convergence within an *order window* between  $k - 1$  and  $k + 1$ . The rationale for the order window is that if convergence appears to occur before column  $k - 1$ , it is often spurious, resulting from some fortuitously small error estimate in the extrapolation. On the other hand, if you need to go beyond  $k + 1$  to obtain convergence, your local model of the convergence behavior is obviously not very good and you need to cut the stepsize and reestablish it.

Here are the steps:

- Test for convergence in column  $k - 1$ : If  $\text{err}_{k-1} \leq 1$ , accept  $T_{k-1,k-1}$ . Set the new target as

$$k_{\text{new}} = \begin{cases} k - 2 & \text{if } W_{k-2} < 0.8W_{k-1} \text{ (order decrease)} \\ k & \text{if } W_{k-1} < 0.9W_{k-2} \text{ (order increase)} \\ k - 1 & \text{otherwise} \end{cases} \quad (17.3.14)$$

Set the corresponding stepsize as

$$H_{\text{new}} = \begin{cases} H_{k_{\text{new}}} & \text{if } k_{\text{new}} = k - 1 \text{ or } k - 2 \\ H_{k-1} \frac{A_k}{A_{k-1}} & \text{if } k_{\text{new}} = k \end{cases} \quad (17.3.15)$$

The idea behind the last formula is that you can't set  $H_{\text{new}} = H_k$  because you're stopping the integration in row  $k - 1$  so you don't compute  $H_k$ . However, since  $k$  is supposedly optimal,  $W_k \approx W_{k-1}$ , which gives the last formula for  $H_{\text{new}}$ .

- If  $\text{err}_{k-1} > 1$ : Check if you can expect convergence by row  $k + 1$  by estimating what  $\text{err}_{k+1}$  will be. Assuming one is in the asymptotic regime, one can show that

$$\text{err}_k \approx \left(\frac{n_0}{n_k}\right)^2 \text{err}_{k-1} \quad (17.3.16)$$

and hence that  $\text{err}_{k+1}$  will be greater than one if approximately

$$\text{err}_{k-1} > \left(\frac{n_k}{n_0}\right)^2 \left(\frac{n_{k+1}}{n_0}\right)^2 \quad (17.3.17)$$

If this condition is satisfied, reject the step and restart with  $k_{\text{new}}$  and  $H_{\text{new}}$  chosen according to (17.3.14) and (17.3.15).

- If (17.3.17) is not satisfied, compute the next row of the tableau (i.e., for the target value of  $k$ ) and see if convergence is attained for column  $k$ . Thus, if  $\text{err}_k \leq 1$ , accept the step and continue with

$$k_{\text{new}} = \begin{cases} k - 1 & \text{if } W_{k-1} < 0.8W_k \text{ (order decrease)} \\ k + 1 & \text{if } W_k < 0.9W_{k-1} \text{ (order increase)} \\ k & \text{otherwise} \end{cases} \quad (17.3.18)$$

Set the corresponding stepsize as

$$H_{\text{new}} = \begin{cases} H_{k_{\text{new}}} & \text{if } k_{\text{new}} = k \text{ or } k - 1 \\ H_k \frac{A_{k+1}}{A_k} & \text{if } k_{\text{new}} = k + 1 \end{cases} \quad (17.3.19)$$

- If  $\text{err}_k > 1$ , check if you can expect convergence by the next row. Analogously to (17.3.17), check if

$$\text{err}_k > \left(\frac{n_{k+1}}{n_0}\right)^2 \quad (17.3.20)$$

If this condition is satisfied, reject the step and restart with  $k_{\text{new}}$  and  $H_{\text{new}}$  chosen according to (17.3.18) and (17.3.19).

- If (17.3.17) is not satisfied, compute row  $k + 1$  of the tableau. If  $\text{err}_{k+1} \leq 1$ , accept the step. Set the new target with the following prescription:

$$\begin{aligned} k_{\text{new}} &= k \\ \text{if } W_{k-1} < 0.8W_k & \quad k_{\text{new}} = k - 1 \text{ (order decrease)} \\ \text{if } W_{k+1} < 0.9W_{k_{\text{new}}} & \quad k_{\text{new}} = k + 1 \text{ (order increase)} \end{aligned} \quad (17.3.21)$$

The stepsize is set as in (17.3.19).

- If  $\text{err}_{k+1} > 1$ , reject the step. Restart with  $k_{\text{new}}$  and  $H_{\text{new}}$  chosen according to (17.3.18) and (17.3.19).

Two important refinements to this strategy are

- After a step is rejected, the order and stepsize are not allowed to increase.
- $H_{\text{new}}$  computed from equation (16.4.5) is not allowed to change too rapidly in one step. It is restricted by

$$\frac{F}{S_4} \leq \frac{H_{\text{new}}}{H} \leq \frac{1}{F} \quad F \equiv S_3^{1/(2k+1)} \quad (17.3.22)$$

The default values of the parameters are  $S_3 = 0.02$ ,  $S_4 = 4$ .

For the first step, the target  $k$  is estimated crudely from the requested precision, but the step is accepted if the error is small enough for any smaller  $k$ . For the last step, the stepsize is decreased to be the length of the remaining integration interval, so a similar increase in the order window is allowed.

### 17.3.4 Dense Output

The basic Bulirsch-Stoer step  $H$  is typically much larger than in Runge-Kutta methods because of the high orders invoked, so a dense output option is even more important. Our implementation once again is based closely on the coding in [4], which is based on [5].

A dense output algorithm turns out to be possible only for certain stepsize sequences, for example increasing by fours:

$$n = 2, 6, 10, 14, 18, 22, 26, 30, \dots \quad (17.3.23)$$

The idea is to do Hermite interpolation using the function and derivative values at the beginning and end of the step. These are supplemented with values of the function and its derivatives at the midpoint obtained by extrapolation of values saved during the integration.

The error of the Hermite interpolation needs to be monitored. If it is too big, the step is rejected and the stepsize reduced appropriately. The error estimate of the interpolation is also used if necessary to limit the size of the next step after a successful step.

### 17.3.5 Implementation

The use of `StepperBS` is exactly the same as the use of the Runge-Kutta routines. For example, to solve the problem at the end of §17.2, everything is exactly the same except the line

```
Odeint<StepperDopr5<rhs_van> > ode(ystart,x1,x2,atol,rtol,h1,hmin,out,d);
```

is replaced by

```
Odeint<StepperBS<rhs_van> > ode(ystart,x1,x2,atol,rtol,h1,hmin,out,d);
```

The object `StepperBS` implements a Bulirsch-Stoer step. Some of its functions are declared virtual because the algorithm `StepperStoerm` will be implemented as a derived class from it in the next section, and these functions will be overridden. As with `StepperDopr5`, the class is templated on the functor class defining the right-hand side derivatives.

```
template <class D>
```

```
struct StepperBS : StepperBase {
```

Bulirsch-Stoer step with monitoring of local truncation error to ensure accuracy and adjust stepsize.

|                                                                                                  |                                                                                                     |
|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>typedef D Dtype;</code>                                                                    | Make the type of <code>derivs</code> available to <code>odeint</code> .                             |
| <code>static const Int KMAXX=8,IMAXX=KMAXX+1;</code>                                             |                                                                                                     |
| <code>KMAXX</code> is the maximum number of rows used in the extrapolation.                      |                                                                                                     |
| <code>Int k_targ;</code>                                                                         | Optimal row number for convergence.                                                                 |
| <code>VecInt nseq;</code>                                                                        | Stepsize sequence.                                                                                  |
| <code>VecInt cost;</code>                                                                        | $A_k$ .                                                                                             |
| <code>MatDoub table;</code>                                                                      | Extrapolation tableau.                                                                              |
| <code>VecDoub dydxnew;</code>                                                                    |                                                                                                     |
| <code>Int mu;</code>                                                                             | Used for dense output.                                                                              |
| <code>MatDoub coeff;</code>                                                                      | Coefficients used in extrapolation tableau.                                                         |
| <code>VecDoub errfac;</code>                                                                     | Used to compute dense interpolation error.                                                          |
| <code>MatDoub ysave;</code>                                                                      | <code>ysave</code> and <code>fsave</code> store values and derivatives to be used for dense output. |
| <code>MatDoub fsave;</code>                                                                      |                                                                                                     |
| <code>VecInt ipoint;</code>                                                                      | Keeps track of where values are stored in <code>fsave</code> .                                      |
| <code>VecDoub dens;</code>                                                                       | Stores quantities for dense interpolating polynomial.                                               |
| <code>StepperBS(VecDoub_IO &amp;yy, VecDoub_IO &amp;dydxx, Doub &amp;xx, const Doub atol,</code> |                                                                                                     |

`stepperbs.h`